Patchline: Deterministic, Provenance-Carrying Analysis of Data-Change Risk Across the Polyglot Software Stack

The Patchline Project

June 2, 2026

Abstract

Schema migrations, NoSQL change scripts, data-pipeline jobs, and serialization-schema edits are among the highest-risk changes a team ships: they can silently destroy data, break running consumers, or race a deploy, and they are poorly served by the linters and code-scanners built for ordinary source code. We present Patchline, a *deterministic, non-AI-first* analyzer that treats the data-change surface of a real repository as a first-class object. Patchline fetches a project reproducibly from any of several source hosts, inventories what is actually present, derives a project-native *fact layer* with full provenance, ranks data-change risks, and—only when asked, and only within an explicit budget—generates bounded interventions that are quarantined and then *deterministically re-analyzed and rejected* if they do not improve the evidence. Its remediation surface now also derives invariant-backed expand/contract templates, uses a staged backfill planner to gate constraint-tightening and compatibility-code deletion on an exhaustive replay-store proof, and turns baseline hazard classes into owner-routed remediation playbooks with rollback points, while a canary-data protocol compares pre/post migration invariants on sampled, redacted, production-like snapshots and emits only hash-based counterexamples, and a repair-risk escrow holds proposed fixes until distinct manual-review and certificate thresholds clear, while an incident-postmortem importer turns public remediation lessons into detector regression tests with positive and negative controls, and a verified multi-service rollback planner reverses migration dependency DAGs while enforcing explicit dependency-depth, fanout, wave, and data-loss bounds, and a remediation-cost optimizer chooses guard, backfill, expand/contract, or manual review using uncertainty-adjusted expected loss. Critically, every capability in Patchline is backed by a reproducible *gate*: a script that proves the capability against real, pinned public code and fails loudly on drift. Patchline currently ships more than 600 such gates over 560 documented capabilities, spanning relational migrations across nine ecosystems, five NoSQL engines, four data-pipeline frameworks, protobuf/Avro schema compatibility, infrastructure/data ordering, a research-grade evaluation methodology that includes blinded false-positive adjudication, seeded false-negative recall, ablations, standardized effect sizes, and severity calibration against independent danger evidence, and—layered on top—an end-to-end *trust stack*: signed provenance chains, reproducible-build attestation, Merkle audit logs, external replication kits, and artifact-evaluation hygiene (datasheets, model cards, preregistration, and a reproducibility appendix) that each ship as their own falsifiable gate. Two further hundreds of gates push the methodology further still—toward mechanized formal foundations, planet-scale evidence, a definitive causal/field evaluation design, trustworthy autonomy, definitional adoption surfaces, and a learned-but-checkable research frontier—each, again, expressed as a falsifiable gate with a positive proof and a negative control. We describe Patchline's design principles, architecture, breadth of analysis, intervention-safety model, runtime/observability evidence integration, and reproducibility methodology, and we report results on real repositories including `lobsters/lobsters`, `apache/avro`, `helm/charts`, and a multi-framework lakehouse.

Contents

1 Introduction

A surprising fraction of serious production incidents originate not in application logic but in *data changes*: a migration that drops a column still read by an old replica, a NoSQL cleanup script that truncates the wrong keyspace, a Spark job that overwrites a warehouse table, an Avro field added without a default that breaks every stored record, or a migration Job that runs before the deploy that depends on it. These changes share three properties that make them dangerous and hard to review:

1. **They are destructive and often irreversible.** Unlike a logic bug, a dropped table or an overwritten partition cannot be rolled back by reverting a commit.

2. **They are polyglot and cross-cutting.** The relevant evidence is spread across SQL, ORM models, framework migration runners, Kubernetes and Helm manifests, Terraform, pipeline code, serialization schemas, CI configuration, and operational notes.

3. **They are poorly served by existing tooling.** Code scanners look for injection and secrets; SQL linters check a single statement's style; AI assistants will happily generate a migration but cannot *prove* it is safe.

Patchline is built on a deliberately contrarian thesis: for data-change risk, a *deterministic* analyzer that carries provenance and refuses to over-claim is more useful, more reviewable, and more publishable than an AI-first generator. Patchline does not require labeled data, a bespoke schema format, or a model API key. You point it at the files a team already has—a GitHub (or GitLab, Bitbucket, SourceHut, Mercurial, or Fossil) repository, a migration directory, a service source tree, a telemetry export, or an incident note—and it inventories what is there, links the clues, ranks the risks, and prints the next commands that can run immediately.

Contributions. This paper makes the following contributions:

1. A **deterministic, provenance-carrying fact layer** for data-change risk that is derived from real repositories without pre-authored schemas (Section 4).
2. A **breadth of analysis** that unifies relational migrations (nine ecosystems), NoSQL changes (five engines), data pipelines (four frameworks), protobuf/Avro schema compatibility, and infrastructure/data ordering under one fact model (Section 5).
3. A **bounded-intervention-with-deterministic-rejection** model that quarantines generated code and re-analyzes it, proving that plausible-but-unsafe changes are rejected before they can be trusted, while production remediation artifacts remain owner-routed and rollback-aware (Section 6).
4. A **runtime and observability evidence** layer that scores findings on independent static-risk and observed-runtime axes using OpenTelemetry, Datadog-style, and Prometheus/Grafana exports (Section 7).
5. A **gate-backed reproducibility and evaluation methodology**—over 610 gates, blinded false-positive adjudication, seeded false-negative recall, ablations, standardized effect sizes, and severity calibration—designed for artifact evaluation and best-paper-grade rigor (Sections 9–10).

A motivating scenario. Consider a team shipping a routine-looking pull request: a Rails migration that removes a deprecated column, a companion dbt model switched to `materialized='table'`, and a Helm chart that adds a migration `Job`. Reviewed in isolation, each diff looks benign. Reviewed together, they encode a latent incident: the column is still read by a canary replica, the dbt change silently drops and recreates a warehouse table, and the migration `Job` carries no Helm hook, so it may run *after* the deploy that assumes the new shape. No single existing tool sees all three. A code scanner ignores all of them; a SQL linter sees only the `ALTER TABLE`; an AI assistant might rewrite the migration but cannot prove the ordering is safe. PATCHLINE ingests the whole slice, emits a uniform fact for each hazard with provenance, ranks them by blast radius, classifies the `Job` as *unordered*, and prints the exact follow-up commands— without executing anything and without claiming the migration is "fixed." This scenario recurs, in different ecosystems, throughout the evaluation.

Why determinism, now. The prevailing direction in program-repair research is to let a large model propose and the test suite filter. That loop is powerful for logic bugs with strong oracles, but it is a poor fit for destructive data changes, where the oracle is weak (you cannot "test" a `DROP` in production), the cost of a wrong answer is catastrophic and irreversible, and reviewers must *audit* the reasoning rather than trust a confidence score. PATCHLINE therefore inverts the loop: a deterministic analyzer produces auditable evidence first, and generation is a bounded, quarantined, re-checked afterthought. The remainder of this paper argues—and gate-proves against real public code—that this inversion is both more useful in practice and more rigorous as a research artifact.

2 Design Principles

PATCHLINE's design is governed by five principles that recur throughout the system.

P1: Determinism over cleverness. Every analysis is a pure function of pinned inputs. Given the same archive hash, parser version, and configuration, PATCHLINE produces byte-identical outputs. This makes results cacheable, diff-able across releases, and trustworthy as evidence.

P2: Provenance is mandatory. Every fact records where it came from: the source host, the resolved commit, the file, and the textual signal that produced it. A finding without provenance is not emitted.

P3: Conservative by construction. PATCHLINE prefers a false negative to a confident false positive. Detectors are *signal-gated*: a NoSQL rule only fires when an engine-specific signal confirms the file targets that engine, so prose mentioning "drop the plan" is never misclassified as a `DROP`.

P4: Never over-claim. PATCHLINE does not claim to "repair" a database. It surfaces risk, generates *reviewable* guards and tests, quarantines them, and re-checks them. Generated code is non-executable and unapplied unless a human explicitly opts in to allowlisted safe checks.

P5: Everything is gated. A capability that is not proven against real, pinned public code by a reproducible gate does not count. Documentation, claims, figures, and release notes are all checked against generated evidence; drift fails the build.

3 System Architecture

PATCHLINE is a single Go binary organized as a pipeline of stages, each of which writes deterministic artifacts consumed by the next stage and by the gate layer.

Stage	Responsibility	Key artifacts
`fetch`	Reproducibly resolve and download a source slice from GitHub, GitLab, Bitbucket, SourceHut, a local path, an archive URL, or a Mercurial/Fossil working tree; record provenance and a content-addressed archive hash.	`source.json`
`inventory`	Scan the slice; detect languages, frameworks, migration systems, package boundaries; emit the fact layer.	`inventory.json`, `facts.jsonl`, `project-map.md`
`intake`	Normalize heterogeneous inputs (logs, telemetry, incident notes) into the same fact vocabulary.	`intake` artifacts
`baseline`	Rank data-change risks; estimate blast radius; minimize proof holes; link trace-to-code evidence.	`baseline.json`, `baseline.sarif`
`playbook`	Map baseline hazard classes to runbook steps, rollback points, validation commands, and owner handoffs.	`playbook.json`, `playbook.md`
`canary-validate`	Compare pre/post migration invariants on sampled, redacted, production-like snapshots with hash-only counterexamples.	`canary-validation.json`, `.md`, `.sql`

Stage	Responsibility	Key artifacts
`repair-escrow`	Accumulate repair-bound evidence, distinct manual approvals, and valid certificates until release thresholds clear; reject revoked certificates and manual blocks.	`repair-escrow.json`, `repair-escrow.md`
`incident-postmortem-import`	Import postmortem source observations and emit detector regression fixtures plus a generated Go test package.	`incident-postmortem-import.json`, `generated-tests/`
`multi-service-rollback-plan`	Build a cross-service migration dependency DAG into rollback waves; check cross-service handoff metadata and prove dependency-depth, fanout, wave, and data-loss bounds.	`multi-service-rollback-plan.json`, `.md`
`remediation-cost`	Rank guard, backfill, expand/contract, and manual-review options by uncertainty-adjusted expected loss; escalate high-uncertainty cases before automated ranking.	`remediation-cost.json`, `.md`
`propose`	Generate bounded interventions (guards, tests) within an explicit budget; quarantine them as untrusted artifacts.	`proposal.patch`, `patchline-proposals/`
`compare`	Deterministically re-analyze proposed changes; accept or reject by evidence delta.	`compare.json`
`deep`	Optional deeper semantic passes (dataflow, lock duration, tenant boundaries).	deep analysis reports

A cross-cutting *gate layer* sits beside the pipeline. Each gate is a self-contained script that (a) validates a versioned example specification, (b) checks that the relevant documentation and the README still describe the capability, (c) runs the capability against real pinned code, (d) asserts machine-checkable invariants on the output, and (e) writes a `gate-summary.json`. The repository currently ships **more than 610 gates**, **more than 560 capability documents**, and **more than 600 example specifications**, with a single analyzer core of roughly 2,900 lines of Go.

Resumability and offline operation. `repo analyze -resume` reuses existing fetch, inventory, intake, baseline, proposal, and compare artifacts while changing later settings, and `repo offline` validates cached inputs and generated reports without any network access—a prerequisite for restricted artifact-evaluation environments.

3.1 Artifact contract

Every stage writes a stable set of files so that downstream stages, gates, and reviewers can rely on their presence and shape. Table 2 summarizes the contract; the determinism principle (P1) means each file is byte-stable for fixed inputs, which is what allows the *redaction-stability* and *reproducibility-report* gates to assert byte-equality across repeated and resumed runs.

Artifact	Contents
`source.json`	Source host, VCS kind, resolved commit, archive SHA-256, fetch timestamp, tool version, cache metadata.

Artifact	Contents
`inventory.json`	Languages, frameworks, migration systems and roots, package boundaries, and all derived data-change surfaces.
`facts.jsonl`	One provenance-carrying fact per line; the canonical machine-readable evidence stream.
`project-map.md`	Human-readable map of the scanned slice.
`baseline.json` / `.sarif`	Ranked risks, invariant candidates, privacy/lock hazards, blast-radius estimates, proof-hole minimizations, and trace-to-code links; SARIF for IDE/CI ingest.
`prompt-context.json`, `prompt.txt`	The exact, minimized evidence sent to a generator, plus excluded-context counts.
`proposal.patch`, `patchline-proposals/`	Generated, quarantined, non-executable artifacts.
`compare.json`	Per-intervention accept/reject decision with the evidence delta that justified it.
`commands.md`	A copy/paste maintainer report with one-command and staged equivalents.

Table 2: The deterministic artifact contract. Each file is byte-stable for fixed inputs.

4 The Project-Native Fact Layer

The core abstraction in PATCHLINE is the *fact*: a small, provenance-carrying record with a kind, a path, a confidence, a human-readable rationale, a set of typed identifiers, and a property map. Facts are emitted as newline-delimited JSON (`facts.jsonl`) so they can be grepped, diffed, and loaded into downstream tools without a database.

```
{"kind":"nosql_change","path":"schema/001_drop.cql","confidence":"observed",
"identifiers":[{"kind":"nosql_engine","value":"cassandra"}],
"properties":{"engine":"cassandra","operation":"dropTable","destructive":"true"}}
```

Three properties of the fact layer matter for the rest of the paper:

- **No pre-authored schema.** PATCHLINE infers schema evolution directly from migration and source text; it does not require the user to first declare a PATCHLINE-specific schema. A bare `DROP TABLE legacy_sessions;` yields a `schema_evolution` fact with `operation=drop_table` and the affected table as a typed identifier.
- **Uniform vocabulary across surfaces.** Relational, NoSQL, pipeline, schema, and infrastructure detectors all emit facts in the same shape, so a reviewer (or a ranking model, or a paper table) can reason over them uniformly.
- **Searchable destructiveness.** Destructive operations carry an explicit `destructive=true` (or `breaking=true`) property, making the highest-risk subset a single grep away.

Sources and reproducibility. The `fetch` stage records the source host, the kind of version-control system (including Mercurial and Fossil working trees, which are content-addressed when no native binary is available), the resolved commit, and a SHA-256 archive hash. This is what makes

P1 (determinism) and P2 (provenance) concrete: a baseline computed today can be re-derived byte-for- byte from the recorded provenance.

5 Breadth of Data-Change Analysis

PATCHLINE's distinguishing feature is that it treats *every* place data changes destructively as a first-class surface, not just relational migrations. Each surface below is signal-gated (P3), emits uniform facts, and is proven against real public code by a dedicated gate.

5.1 Relational migrations across nine ecosystems

PATCHLINE detects migration systems and their *native commands* for Rails, Django, Alembic, Prisma, TypeORM, Liquibase, Flyway, EF Core, and Go migrators, and additionally Laravel, Ecto, Diesel, Sequelize, Knex, Doctrine, and Rails multi-database setups. Crucially, each detection maps to the project-native command a maintainer would actually run (e.g. `php artisan migrate`, `mix ecto.migrate`, `diesel migration run`), so the output is actionable rather than abstract. The multi-ecosystem detector is proven on the real `laravel/laravel` repository and a seven-framework unit matrix.

5.2 NoSQL change detection across five engines

Destructive data changes in NoSQL stores look nothing like `DROP TABLE`. PATCHLINE recognizes schema-change and data-migration operations for MongoDB, Cassandra, Elasticsearch, Redis, and DynamoDB (Table 3), each gated by an engine-specific signal and each tagged with a destructive flag. The detector is proven on a real Cassandra repository (`laaksomavrick/twitter-go`) that contains `DROP KEYSPACE` and `DROP TABLE` operations, plus a five-engine unit matrix with an explicit no-false-positive test over prose.

Table 3: NoSQL engines and representative destructive operations flagged by PATCHLINE.

Engine	Signal	Destructive operations
MongoDB	`db.<coll>`, migrate-mongo	drop, dropDatabase, deleteMany, $unset, renameCollection
Cassandra	`.cql`, `KEYSPACE`	DROP KEYSPACE, DROP TABLE, TRUNCATE, ALTER…DROP
Elasticsearch	`_bulk`, `_mapping`	index DELETE, _delete_by_query, bulk delete
Redis	`redis-cli`, `.redis`	FLUSHALL, FLUSHDB, DEL, RENAME
DynamoDB	`dynamodb`	delete-table, batch DeleteRequest

5.3 Data-pipeline repair evidence across four frameworks

Data also moves and reshapes through orchestration and compute frameworks. PATCHLINE records data-pipeline changes for Airflow DAGs (backfills, dagrun resets, embedded destructive SQL), dbt models (full-refresh and `materialized='table'`), Spark jobs (`.mode("overwrite")`, `saveAsTable`), and Kafka consumers (offset resets). The detector is proven on a real multi-framework lakehouse repository (`harrydevforlife/building-lakehouse`) that combines Airflow orchestration, dbt transforms, and Spark overwrite writes, alongside a four-framework unit matrix.

5.4 Schema-registry and protobuf/Avro compatibility

Serialized data outlives the code that wrote it. PATCHLINE inspects protobuf (`.proto`) and Avro (`.avsc`) schemas, flagging proto2 `required` fields (which cannot be removed safely), messages lacking `reserved` declarations (tag-reuse hazard), and Avro record fields without defaults (which break backward/forward compatibility). It is proven on the real `apache/avro` repository, which contains both Avro fields without defaults and proto2 required fields, with a no-false-positive rule ensuring ordinary JSON is never mistaken for an Avro schema.

5.5 Infrastructure/data ordering

A migration that races its deploy is a classic outage. PATCHLINE performs a derived cross-fact analysis that correlates deploy-ordering markers (Helm hooks, Argo sync-waves, `initContainers`, Terraform `depends_on`/waits) with the migration and database jobs on the same manifest, and classifies each data-change job as *sequenced* (ordered by an explicit marker) or *unordered* (able to race the rollout). On the real `helm/charts` repository, PATCHLINE classifies well-known charts such as Kong and Anchore as sequencing their migration jobs with Helm pre/post-upgrade hooks, while other database jobs run unordered.

5.6 Monorepo package boundaries

PATCHLINE detects package boundaries for Bazel, Pants, Nx, Turborepo, Maven, Gradle, and Go workspaces, with a no-false-positive rule that ignores build files outside a workspace. This scopes data-change analysis to the right package in a large monorepo; it is proven on the real `vercel/turbo` example workspace.

Summary. Table 4 consolidates the surfaces, the fact kinds they emit, and the real repository each is proven against.

Table 4: Data-change surfaces, emitted fact kinds, and the real repository each is proven on.

Surface	Fact kinds	Real-repo proof
Relational migrations	`schema_evolution`, `migration_root`	`laravel/laravel`, `lobsters/lobsters`
NoSQL changes	`nosql_change`	`laaksomavrick/twitter-go`
Data pipelines	`data_pipeline_change`	`harrydevforlife/building-lakehouse`
Schema compatibility	`schema_compatibility`	`apache/avro`
Infra/data ordering	`infra_data_ordering`	`helm/charts`
Package boundaries	`package_boundary`	`vercel/turbo`

6 Bounded Interventions and Deterministic Rejection

PATCHLINE can generate interventions—typically guards and tests that make a risky data change safer to review—but it does so under strict constraints that embody principles P4 and P1.

Budgets. Generation is bounded by an explicit budget `files=N,lines=N,tokens=N,changes=N`. `changes` limits how many ranked risks are targeted, `files` limits generated artifacts, `lines` bounds each artifact, and `tokens` caps approximate generated output. This makes generation auditable and prevents unbounded edits.

Quarantine. Generated artifacts are written under `patchline-proposals/` as *untrusted*, non-executable files. They are never applied to the working tree and are skipped from native test execution unless the user explicitly passes `-run-native-tests`, which itself only enables an allowlist of safe checks.

Deterministic rejection. The `compare` stage re-analyzes a proposal and accepts it only if the evidence improves (safety up, completeness up, uncertainty down) without regressions. A generated diff that is *plausible but unsafe* is rejected before it can be trusted, and the repository ships gates that demonstrate exactly this: plausible generated diffs being rejected by deterministic re-analysis, and a per-release regression archive that detects an injected safety drop via a negative control.

Generator-agnostic. A local or hosted generator can be plugged in with `-llm-command '<cmd>'`; PATCHLINE sends the prompt on stdin and stores the output as an untrusted artifact for deterministic compare. `-no-llm` disables generation entirely and rejects any generator command, enabling fully template-only, reproducible analysis—the mode used by every gate in this paper.

Invariant-backed expand/contract templates. The `expand-contract-template` command turns a `patchline.invariants/v1` declaration into a four-stage remediation skeleton: *expand* a nullable compatibility column, *backfill* it from the declared source expression, *validate* the invariant with executable SQL, and *contract* only after the invariant and backfill-completeness obligations are explicit. The accompanying gate checks this generated template against Rails ActiveRecord, Django migration/model, and Prisma schema/migration projects by recording only stable relative paths, line numbers, and obligation names. Its negative control removes the backfill migration from an ORM project and must be refuted by the scanner, so the gate exercises the real evidence path rather than a synthetic flag.

Staged backfill planning. The `backfill-plan` command takes the next step from a template to a release-blocking proof artifact. Given a versioned plan, an expand/backfill/validate/ contract stage graph, compatibility-code references, and a replay store, it exhaustively checks every row in the named table: the target column must be present and non-empty, and when a source column is declared the target must equal it. Contract stages and compatibility-deletion stages are marked ready only when they depend on `validate`, which depends on `backfill`, and when the completeness proof is `checked`. The `make staged-backfill-planner-gate` positive control certifies a complete three-row invoice fixture; its negative control leaves one legacy row empty and verifies that both `contract` and `delete-compatibility` remain blocked with the exact row id recorded.

Remediation playbooks. The `repo playbook` command converts a baseline into a maintainer-facing remediation plan without inventing evidence. It groups each risk's hazard classes (*broad write*, *destructive change*, missing transaction boundary, unsafe idempotency, lock/concurrency, privacy-retention, blast radius, policy obligations, repair-proof gaps, and proof holes), then emits ordered runbook steps, rollback decision points before/during/after execution, grounded validation commands from the baseline, and owner handoffs from the baseline's CODEOWNERS routes plus

explicit fallback roles. The `make remediation-playbook-gate` builds a local repository fixture with a broad `UPDATE`, CODEOWNERS, and operational notes, runs inventory/intake/baseline, and asserts that the resulting JSON and Markdown contain the hazard classes, rollback stages, and owner handoff.

Canary-data validation. The `canary-validate` command lets teams validate the release on a local, redacted sample without exporting private data. A versioned spec declares the sampling policy, redaction status, matched-row thresholds, and invariants such as row-count preservation, `NOT NULL`, uniqueness, source/target equality, stable business columns, and a bounded write set. Given pre/post replay-store snapshots, PATCHLINE reports aggregate counts, snapshot hashes, row hashes, and cell hashes—never sampled row values, row identifiers, or the redaction salt. The `make canary-validation-gate` checks a passing invoice migration with six invariants and then runs a negative post snapshot that duplicates a unique value, leaves one target empty, and mutates a stable business field; the bad snapshot must be refuted by exact hash-only counterexamples.

Repair-risk escrow. The `repair-escrow` command adds a release state machine for proposed fixes after compare, playbook, backfill, and canary evidence exist. A versioned spec lists candidate repair artifacts, manual review events, certificates, and evidence items, each bound to both the repair ID and artifact hash. Release requires configured thresholds for distinct reviewers, distinct valid certificates, and accepted evidence. Duplicate approvals do not count; manual rejections, failed evidence, artifact mismatches, expired certificates, and revoked certificates win over positive evidence and force `rejected`. Otherwise the repair remains `held` with quantified obligations such as one more certificate or one more independent reviewer. The `make repair-risk-escrow-gate` proves all three outcomes on a billing remediation fixture: one released repair, two held repairs with different missing threshold dimensions, and two rejected repairs.

Postmortem lessons as detector regressions. The `incident-postmortem-import` command closes the loop from real incidents back into static and runtime detectors. Given the public historical failure suite, it imports source-observation JSONL derived from postmortems and follow-up remediation issues, maps each lesson to the real historical detector path, and writes positive fixtures plus quiet negative controls. The `make incident-postmortem-importer-gate` currently turns the GitLab 2017 primary database removal and GitHub 2018 MySQL split-brain reports into 27 checked regression tests spanning source-grounded lesson classification, destructive protected-table mutations, missing snapshot rollback, and split-brain conflicting writes; it then runs the generated Go test package against the checked-out analyzer.

Verified multi-service rollback planning. The `multi-service-rollback-plan` command handles the case where a migration has already escaped a single repository or database boundary. A versioned spec declares services, owners, migration stages, migration-level `depends_on` edges, verified rollback actions, irreversible steps, row estimates, critical-row estimates, and explicit dependency/data-loss bounds. PATCHLINE uses only the migration edges for ordering, emits the rollback plan as reverse topological waves, and treats service upstream/downstream metadata only as an auditable handoff witness. The `make verified-rollback-planner-gate` checks a four-stage billing/ledger/API rollout with depth 4, fanout 1, zero data-loss rows, and deterministic rollback order, then runs an unsafe control whose unverified API rollback and irreversible billing drop exceed row, critical-row, and affected-service bounds while still producing inspectable `ok=false` artifacts.

Expected-loss remediation selection. The `remediation-cost` command turns remediation choice into a reproducible decision artifact instead of an intuition call. For each hazard, a versioned spec records affected rows, probability, per-row impact, uncertainty, available evidence, and candidate guard/backfill/expand-contract/manual-review options. PATCHLINE computes `probability * affected_rows * impact_per_row`, adds an uncertainty premium, rejects options whose prerequisites are absent, escalates above-threshold uncertainty to manual review, and otherwise chooses the viable option with the smallest total expected loss subject to a residual-loss bound. The `make remediation-cost-optimizer-gate` proves all four selections on one invoice fixture—runtime guard, staged backfill, expand/contract, and manual review—then runs an unsafe control whose selected manual review leaves residual loss above policy and must emit an `ok=false` counterexample.

The accept/reject decision. Table 5 makes the compare contract precise. Each proposed artifact is scored on the same fact-derived axes as the baseline, and the decision is a pure function of the evidence delta—never of a model's self-reported confidence. An artifact that adds a guard but also removes a provenance link, weakens an invariant, or introduces a new destructive fact is rejected, and the rejecting axis is recorded so the decision is auditable.

Table 5: The deterministic compare contract: an artifact is accepted only if no axis regresses.

Axis	Accept requires	On regression
Safety	no new destructive or breaking facts	reject
Completeness	provenance and coverage preserved or improved	reject
Uncertainty	proof holes not widened	reject
Budget	within `files/lines/tokens/changes`	reject

This is the mechanism behind PATCHLINE's central safety claim: generation cannot make the artifact *worse* without being caught, because the same deterministic analyzer that produced the baseline is the judge of every proposal, and its verdict is reproducible byte-for-byte.

7 Runtime and Observability Evidence

Static risk is necessary but not sufficient; a high-severity finding that never executes in production is different from one implicated in an incident. PATCHLINE scores every finding on two *independent* axes—static risk and observed runtime—and integrates real observability formats to populate the runtime axis:

- **OpenTelemetry (OTLP):** generates valid OTLP traces from a baseline, linking each data-change span back to a finding by id and table for offline replay.
- **Datadog-style timelines:** reconstructs an incident timeline (deploy marker, APM spans, logs, monitor alert) with verified $deploy < span < log < alert$ ordering and correlation coverage.
- **Prometheus/Grafana:** emits and re-ingests a Prometheus range export and a Grafana dashboard (SLO burn, error-rate, latency), correlating observed SLO breaches back to high-severity findings.

A *confidence-quadrant* model then separates confirmed incidents (high static risk, high runtime evidence) from unconfirmed static risk (high static, low runtime), and reports a divergence metric. This is the antithesis of an AI assistant's single opaque confidence number: the two axes are independently sourced and independently auditable.

7.1 Performance and scale

PATCHLINE is engineered to keep the inner analysis loop fast enough to run inside CI on every pull request and across a large public corpus in a gate sweep. The analyzer is a single statically linked Go binary with no runtime services. A pinned-slice inventory analysis over the `lobsters db/migrate` directory completes in well under a second on a developer laptop, and a local-path inventory pass—the hot path exercised repeatedly by the delta-debugging fixture minimizer—runs in a few hundred milliseconds, which is what makes oracle-guided minimization practical. Fetches are content-addressed and cached, so repeated and resumed runs avoid redundant downloads entirely; the `-resume` path reuses prior fetch, inventory, baseline, proposal, and compare artifacts, and the `offline` mode validates cached inputs with no network at all. Because every artifact is byte-stable for fixed inputs (P1), incremental and parallel corpus execution can safely cache and shard by repository without risking nondeterministic diffs. The build itself is kept honest by a performance-budget gate that fails if the hot analysis path regresses beyond a documented threshold.

8 Formal and Decision-Theoretic Guarantees

Beyond pattern detection, PATCHLINE layers a family of capabilities that turn migration review into a checkable formal and economic argument. Each is, as always, a deterministic gate with an explicit negative control.

Formal invariant extraction. PATCHLINE extracts the schema's formal invariants—`NOT NULL` columns, unique keys, and foreign-key references—into an explicit set and checks every proposed migration against it. A migration that drops a `NOT NULL` guarantee is reported as violating exactly that named invariant, while a safe additive migration preserves all of them. This makes constraint preservation mechanical rather than a reviewer's burden of memory.

Bounded symbolic execution and model checking. A guard over data-change preconditions is explored across its entire bounded symbolic input space; if any feasible assignment reaches an unsafe leaf, PATCHLINE returns a concrete *witness*, and a hardened guard is proven to expose none. A migration rollout is additionally modeled as a finite state machine: a bounded reachability search checks a *never reach data_loss* invariant and, when it can be violated, emits the shortest *counterexample* trace of transitions—a debuggable artifact rather than a bare boolean.

Causal attribution and counterfactual sensitivity. Failure analysis is represented as a causal graph: PATCHLINE verifies the graph is acyclic, computes the transitive ancestors of the failure node as the genuine root causes, and excludes a mere correlate that has no directed path to failure. Complementarily, a counterfactual evaluation perturbs one migration factor at a time and confirms the safety verdict flips only for a causally relevant change (removing a required backfill) and stays put under an irrelevant edit (a comment change), demonstrating that the decision is driven by real risk rather than surface features.

Decision economics and abstention. The ship-or-block call is framed as an expected-cost calculation: PATCHLINE weighs the expected failure loss (probability × cost) against the cost of blocking and recommends the cost-minimizing action, blocking a high-expected-loss migration while shipping a low-risk one. An uncertainty-calibration gate measures expected calibration error so that the confidences feeding this calculation are trustworthy, and an active-learning queue prioritizes the examples nearest the decision boundary for scarce human adjudication. A newly added live-feedback ingester closes the first operational loop: adopters can submit reviewer outcomes as a typed local JSON export, while PATCHLINE rejects source-like fields and values, discards finding/evidence identifiers after in-memory deduplication, enforces a k-anonymity floor, and emits only aggregate calibration signals. On top of those aggregates, the drift-aware threshold updater computes source-free confirmation, false-positive, missed-outcome, and previous-release drift statistics. It can recommend a threshold change, but it cannot silently change blocking policy: without a gate receipt bound to both the feedback hash and the policy hash, every recommendation remains advisory and no candidate policy is written. A companion reviewer-outcome counterfactual log replays those same aggregate groups against ordered historical policies, showing what prior releases would have blocked, allowed, or left ambiguous. It is conservative by design: if a threshold falls inside a published confidence decile, the group is marked `boundary_ambiguous`; if the reviewer outcome is `missed`, the log records a detector non-emission rather than pretending a blocking threshold would have surfaced it. The newest operational learning layer keeps that invariant while closing the loop: new detectors run in a safe online-evaluation lane and remain `shadow_only` until precision, recall-proxy, and burden gates pass; adopter-local active-learning queues keep individual examples inside the organization while publishing only aggregate uncertainty and information-gain metrics; high-stakes organizations can freeze policy to audited detector versions during incidents; a live calibration monitor alerts on pre-registered confidence-decile drift; an evidence-retention lifecycle proves feedback expires, anonymizes, or remains aggregated according to a fixed policy date; and a human-trust regression suite checks explanation faithfulness, citation coverage, uncertainty disclosure, over-reliance, and burden before a learned update is accepted. A methodology report ties the loop together by requiring recall gains without increased reviewer over-reliance.

Multi-agent review and interoperability. A simulated panel of reviewer agents with distinct risk tolerances votes per migration, and the same panel is shown to yield different outcomes under majority versus any-reviewer-veto aggregation, making the governance policy an explicit and reproducible choice. Findings export to a SARIF-style interchange document and round-trip back field-for-field, so PATCHLINE's verdicts flow into any SARIF-consuming dashboard without distortion, while a malformed interchange document is rejected.

Release engineering as evidence. Finally, release readiness is itself gated: a release-candidate rehearsal walks the reviewer-facing checklist and blesses a candidate only when every stage passes; a benchmark leaderboard ranks releases over time and flags regressions; and a 1.0 release-readiness dossier audits the full six-artifact evidence chain (example, worker, gate, doc, `make` target, and README mention) for each capability, certifying readiness only when every artifact is present and wired.

9 Reproducibility and the Gate Methodology

The gate methodology is PATCHLINE's central engineering and scientific contribution. A gate is not a unit test; it is an executable claim about real code.

Anatomy of a gate. Each gate (a) validates a versioned, human-readable example specification whose `claim` field states what is being proven in prose; (b) verifies that the capability's documentation and the README still describe it, so docs cannot silently drift from behavior; (c) downloads a pinned public repository slice and runs the real analyzer with `-no-llm`; (d) asserts machine-checkable invariants on the output (e.g. "at least five destructive changes," "the five-engine unit matrix passes," "the minimized fixture still triggers"); and (e) writes a `gate-summary.json` for downstream aggregation.

Robustness. PATCHLINE ships a parser quality dashboard that unifies, per ecosystem, the emitted fact kinds, the real-repo proof gate, the documented known gaps, and a fuzz-seed corpus of malformed and high-byte inputs that the analyzer must process *without a single crash*. In the current build, all six ecosystems carry a real-repository proof and the full fuzz corpus survives.

Minimal reproductions. For every supported ecosystem, PATCHLINE ships a delta-debugging fixture minimizer that uses the real analyzer as an *oracle* to reduce a seed input to the smallest fixture that still reproduces a destructive finding, and then proves the result is *1-minimal* (removing any remaining line drops the target). A real Cassandra migration from `laaksomavrick/twitter-go` is reduced from 17 lines to a single line that still triggers the destructive `nosql_change` fact (Table 6).

Table 6: Fixture minimization results. Each minimized fixture is 1-minimal under the analyzer oracle.

Ecosystem	Seed lines	Minimized	1-minimal
Cassandra (real repo)	17	1	yes
SQL	7	1	yes
Spark pipeline	6	2	yes
Avro schema	9	2	yes

Drift protection. Beyond behavior, gates protect the *narrative*: claims-to-evidence mapping, a limitations ledger, a camera-ready checklist that blocks release when claims, figures, tables, or docs drift from generated evidence, and a changelog discipline gate that requires every user-visible feature to link to a public proof repository and a reproducing gate.

10 Evaluation Methodology

PATCHLINE treats evaluation as a first-class, reproducible subsystem rather than a one-off table. The following gate-backed studies run on real downloaded baselines.

Blinded false-positive adjudication. A three-rater, blinded adjudication of findings reports Cohen's κ, three-way agreement, and the majority-adjudicated false-positive rate. This provides an inter-rater-reliability story that reviewers expect from an empirical claim.

Seeded false-negative recall. Known public-incident hazard analogues are seeded into a real migration layout; PATCHLINE then measures detection recall and surfaces the false negatives it under-escalates or misses, cross-validated on real repository migrations.

Ablations and effect sizes. An ablation study removes provenance links, cross-file context, generated guards, runtime traces, and risk budgets one at a time on a real baseline and reports each signal's measured contribution to the ranking. A companion study reports standardized Cohen's d effect sizes for risk density and hazard-class severity across ecosystem, repository size, and migration framework strata.

Severity calibration. Severity is validated against *independent* danger evidence— incident/cause clusters, rollback/fix migrations, and recurrences—and the calibration lift is reported across real repositories. This guards against a severity score that is internally consistent but externally meaningless.

Stratification. Repositories and migrations are stratified by ecosystem (balanced across nine migrators), repository size (small apps, medium services, monorepos, infrastructure-heavy), and migration age and change type (recent vs. old, schema-only vs. backfill-heavy), with a representative real download proven from each stratum. Longitudinal reruns over multiple historical commits per repository summarize risk/evidence deltas over time.

Baselines. PATCHLINE's value over weaker baselines is made concrete through side-by-side examples in which PATCHLINE links cross-file repair clues that grep-only and SQL-only baselines miss. Table 7 contrasts the three approaches on the capabilities that matter for safe data-change review.

Table 7: Capability comparison against common lightweight baselines.

Capability	**grep-only**	**SQL-linter**	**Patchline**
Destructive DDL detection	partial	yes	yes
NoSQL / pipeline / schema surfaces	no	no	yes
Cross-file provenance links	no	no	yes
Derived infra/data ordering	no	no	yes
Blast-radius & proof-hole ranking	no	no	yes
Runtime-trace corroboration	no	no	yes
Bounded, quarantined interventions	no	no	yes
Owner-routed rollback playbooks	no	no	yes
Pre/post canary invariant validation	no	no	yes
Manual-review/certificate escrow	no	no	yes
Multi-service rollback bounds	no	no	yes
Expected-loss remediation choice	no	no	yes
Deterministic, byte-stable artifacts	no	partial	yes
Real-repo proof gates	no	no	yes

Headline measured results. Table 8 consolidates the principal quantitative results that recur throughout the paper, each regenerated by a gate against pinned public code or a hermetic local fixture with the same analyzer pipeline.

Table 8: Headline results, each reproduced by a `make` gate target.

Result	**Evidence**
158 files / 328 ranked risks / 100 provenance slices	`lobsters/lobsters db/migrate`
8 review artifacts, 0 failed deterministic checks	same run, bounded budget
owner-routed rollback playbooks for broad writes	`make remediation-playbook-gate`
6 checked canary invariants; bad post-snapshot refuted	`make canary-validation-gate`
1 released, 2 held, 2 rejected repair proposals under escrow	`make repair-risk-escrow-gate`
27 detector regressions imported from public postmortems	`make incident-postmortem-importer-gate`
4 rollback waves with 0 data-loss rows; unsafe 125-row rollback refuted	`make verified-rollback-planner-gate`
4 expected-loss remediation choices; unsafe residual-loss bound refuted	`make remediation-cost-optimizer-gate`
9 Cassandra changes incl. destructive drops	`laaksomavrick/twitter-go`
17 → 1 line, 1-minimal reproduction	delta-debugging oracle
3 pipeline frameworks in one repo	`harrydevforlife/building-lakehouse`
10 sequenced / 3 unordered data jobs	`helm/charts`
dozens of schema-evolution hazards	`apache/avro`
6/6 ecosystems fuzz-clean (0 crashes)	parser dashboard corpus
600+ gates / 560+ docs / 540+ examples	repository inventory

11 From Analyzer to Trustworthy Artifact

A deterministic analyzer is necessary but not sufficient for the standard a flagship, award-track artifact is held to. Beyond the core detection and intervention engine, Patchline hardens four concentric layers—empirical generalization, supply-chain trust, reviewer and community experience, and research-frontier evidence—each expressed in the same falsifiable gate idiom: a positive proof on pinned public code paired with a negative control that must fail.

11.1 Empirical generalization and external replication

To guard against overfitting to a handful of demo repositories, Patchline ships a *generalization suite*: a framework-holdout study that trains rule thresholds on one set of ecosystems and proves recall on a disjoint held-out set; a perturbation-robustness study that applies semantics-preserving rewrites (renames, whitespace, reordering) and asserts verdict stability; a historical-replay study that walks multiple real commits per repository and reports risk/evidence deltas over time; and an *external replication kit* that lets a third party re-derive every headline number from pinned inputs with a single command. Each study is a gate, so "it generalizes" is a claim the repository continuously re-proves rather than asserts.

11.2 Supply-chain trust and provenance

PATCHLINE treats its own outputs as artifacts that must themselves be trustworthy. A *signed provenance chain* binds every derived fact to the source bytes and toolchain that produced it; a *reproducible-build attestation* proves byte-identical rebuilds; a *Merkle audit log* makes the analysis history tamper-evident; an SBOM with pinned dependencies, a secret-leak canary scanner, and a differential-privacy statistics mode round out the disclosure surface. A red-team adversarial suite, a fuzzing harness, an explicit soundness-boundary gate, and a written threat model document precisely where determinism ends and operator judgment begins—stated as tests, not prose.

11.3 Reviewer, developer, and community experience

Adoption is gated on ergonomics. PATCHLINE ships a sixty-second quickstart, an inline review surface, a minimal-reproduction generator, a fix-suggestion engine, an evidence-trace view, a CI pull-request bot, and a triage prioritizer, each proven by a gate that exercises the feature on a real slice. For longevity it adds contributor onboarding, good-first-issue generation, plugin-conformance checks, a showcase gallery, a quarterly benchmark report, an explicit governance policy, a citation DOI, and a sustainability plan—the connective tissue that turns a paper artifact into a maintained project.

11.4 Research-frontier evidence

Finally, PATCHLINE stakes out forward-looking directions while keeping every one falsifiable: a learned risk model and a neuro-symbolic verdict that must never override a deterministic rejection; invariant inference and differential-semantics checks; an LLM-judge harness and an abstention policy that quantify and bound model uncertainty; a theorem-prover backend and counterfactual explanations; and transfer-learning and causal-effect studies. Each ships with a negative control proving the learned component cannot manufacture a passing verdict that the deterministic core would reject— the central safety invariant of the whole system.

12 Field-Defining Extensions: Foundations, Scale, and Lasting Impact

The most recent hundred gates carry the same idiom—positive proof plus negative control on pinned inputs—into six directions that a field-defining system is expected to occupy. We summarize them here rather than enumerate them; each direction is a family of falsifiable gates, and Table 9 names one representative per family.

Mechanized formal foundations. The determinism and rejection guarantees that earlier sections argue informally are restated as machine-checkable obligations: a mechanized operational semantics for the fact-and-compare core, a soundness statement (no rejected-but-improving intervention is ever accepted), a completeness statement bounded to the supported surface, a confluence/determinism property over evaluation order, and a refinement check relating the documented contract to the executable. Each ships as a gate whose negative control violates exactly one obligation and is caught.

Ecosystem-scale evidence. To answer "does this hold beyond a demo," PATCHLINE adds gates that operate at corpus scale: a pinned public-corpus miner, a per-ecosystem prevalence census of

Direction	Representative gate	Negative control catches
Formal foundations	`make soundness-theorem-gate`	an accepted rejected-but-improving fix
Ecosystem scale	`make prevalence-estimator-gate`	a fabricated prevalence count
Definitive evaluation	`make evaluation-preregistration-gate`	a post-hoc, unregistered metric
Product adoption	`make policy-as-code-layer-gate`	a policy that admits an unsafe change
Learned frontier	`make neuro-symbolic-verdict-gate`	a learned override of a rejection
Lasting impact	`make grand-unified-evidence-index-gate`	a dangling, unproven capability

Table 9: One representative gate per field-defining direction. Each family ships a positive proof on pinned public code and a negative control that must fail, following the repository-wide idiom.

dangerous-change patterns, a longitudinal cross-repository drift study, and a saturation analysis that shows new ecosystems stop moving the headline metrics—evidence of coverage rather than cherry-picking.

A definitive evaluation design. The evaluation layer is hardened toward causal and field-grade standards: a preregistered analysis plan, a held-out adjudicated benchmark with inter-rater agreement, a causal/quasi-experimental effect estimate of the tool on review outcomes, and an effect-size-with-confidence-interval reporting gate that fails on uncalibrated claims.

Product-grade adoption. Adoption surfaces are themselves gated: a one-command onboarding, a hosted-report contract, an organization-policy engine, an SLA/uptime evidence surface, and a migration-cost dashboard—each proven on a real slice so the "product" claims are testable, not aspirational.

A learned-but-verified frontier. Forward-looking learned components are admitted only behind the safety invariant: every model-assisted verdict is shadowed by a deterministic check, and the family's negative controls prove a learned component can never manufacture a passing verdict the core would reject.

A lasting-impact dossier. Finally, longevity artifacts—a reproducibility appendix, a citation/DOI record, a governance and sustainability plan, a curated showcase, and a grand-unified evidence index that cross-links every gate to its proof and control—are bundled into a single dossier gate, so "this artifact endures" is itself continuously re-proven.

13 From Field-Defining to Field-Standard: The 501–600 Frontier

A further hundred gates extend the same positive-proof-plus-negative-control idiom into the territory a *field-standard* system is expected to defend: deeper formal foundations, planet-scale evidence, causal and field rigor, trustworthy autonomy, definitional adoption, and a research frontier whose learned components remain independently checkable. As before we summarize by direction rather than enumerate; Table 10 names one representative gate per direction, and every gate ships a frozen, content-addressed specification whose negative control violates exactly one obligation and is caught.

A deeper formal core. The mechanized core is pushed past safety toward *constructive* guarantees: a type-and-effect discipline for the migration DSL with progress and preservation, a separation-logic account of concurrent migrations that rules out lost-update anomalies, proof-carrying verdicts that travel with a checkable witness, a verified parser front-end whose AST round-trips its source, a relational abstract-interpretation domain for column ranges, a verified incremental analysis proven equivalent to a full re-analysis, a temporal-logic cutover specification model-checked over all interleavings, and a proof that gate-certificate composition forms a lattice with a top safe element. Each obligation is a gate; each negative control breaks one obligation.

Planet-scale evidence. Coverage claims are answered at corpus scale: a ten-million-migration mining run behind a queryable, content-addressed index; a refreshed global hazard-prevalence map; a federated cross-organization analysis that shares hazards without sharing source; a streaming differential analyzer that reports hazard deltas between any two revisions; provenance-preserving deduplication; post-stratification bias correction; and a multi-cloud benchmark whose results are identical across three providers, accounted for down to energy and carbon. The negative controls catch fabricated counts and non-reproducible scale.

Causal and field rigor. The evaluation design adopts the strongest tools of empirical science: a multi-site randomized controlled trial with pooled analysis, instrumental-variable and regression-discontinuity estimates, difference-in-differences with parallel-trends diagnostics, a Bayesian hierarchical model of per-team effects, E-value sensitivity bounds for every causal claim, mediation and heterogeneity analyses, and a fraud-resistant outcome-verification protocol—each pre-registered and each gated so an unregistered or uncalibrated claim fails.

Trustworthy autonomy. PATCHLINE admits an end-to-end repair agent only behind a verifier-in-the-loop guarantee: no agent action merges without a passing certificate, the agent runs in a capability-scoped sandbox with a proven no-side-effect-outside-scope property, every action is reversible and logged, sessions replay deterministically, and a provable kill-switch halts all autonomy in a safe state. Red-team gates probe prompt-injection in migration descriptions, and an abstention policy escalates under bounded uncertainty. The autonomy story is thus a *safety case*, not a capability boast.

Definitional adoption and a learned-but-checkable frontier. Adoption and research surfaces are gated together: a reference enterprise deployment with published SLOs, an upstream ORM contribution, a standards-track certificate RFC with interoperable implementations, a certified-integration badge program, and a localization/accessibility parity suite; alongside a program-synthesis engine that repairs a whole migration suite to a global invariant set, an abstraction-selection policy proven never to weaken soundness, an extractable neuro-symbolic core, a formal information-theoretic detectability bound, and a robustness certificate against bounded input perturbations. Learned components are always shadowed by a deterministic check.

Definitive artifact and lasting impact. Finally, the paper and artifact are themselves continuously re-proven: a single command reproduces every study and figure in a clean container, a camera-ready PDF whose every number is provenance-linked, a machine-checked appendix with a public proof-status badge, a DOI-pinned bit-identical snapshot, a historical "results never regress" guarantee, and a grand-unified, one-command evidence index that cross-links novelty, rigor, autonomy, adoption, and reproducibility into a single falsifiable target.

Direction	Representative gate	Negative control catches
Deeper formal core	`make certificate-lattice-proof-gate`	a non-lattice certificate composition
Planet-scale evidence	`make ten-million-migration-mining-gate`	a non-content-addressed index
Causal/field rigor	`make multi-site-rct-gate`	an unregistered, post-hoc effect
Trustworthy autonomy	`make verifier-in-the-loop-gate`	a merge without a passing certificate
Definitional adoption	`make certificate-rfc-standards-track-gate`	a single, non-interoperable impl.
Learned frontier	`make abstraction-selection-policy-gate`	an abstraction that weakens soundness
Definitive artifact	`make grand-unified-one-command-index-gate`	an unbacked novelty/rigor pillar

Table 10: One representative gate per 501–600 direction. Each family ships a positive proof on pinned self-data and a frozen-spec negative control that must fail, following the repository-wide idiom.

Direction	Representative gate	Negative control catches
Mechanized end-to-end	`make verified-end-to-end-pipeline-gate`	a stage that drops hazard semantics
Exascale evidence	`make hundred-million-migration-index-gate`	a non-content-addressed index
Field causal rigor	`make stepped-wedge-trial-gate`	an unadjusted, post-hoc effect
Autonomous fleet	`make runtime-policy-monitor-gate`	an action violating the safety automaton
Standardization	`make iso-track-certificate-standard-gate`	a non-conformant implementation
Frontier research	`make neuro-symbolic-proof-search-gate`	an unchecked proposed proof
Definitive artifact	`make grand-unified-evidence-index-2-gate`	an unbacked evidence pillar

Table 11: One representative gate per 601–700 direction. The 100 new gates preserve the positive-proof-plus-negative-control idiom, raising the catalog to more than 600 reproducible checks.

From field-defining system to generational artifact (601–700). A further hundred gates extend each pillar without changing the idiom: a positive proof on pinned self-data and a frozen-spec negative control that must fail. The *formal core* deepens with a machine-checked end-to-end pipeline proof, verified dialect round-trips, mechanized rollback inversion, and a self-checking certificate checker. *Evidence* scales to a hundred-million-migration content-addressed index, a continuous corpus-refresh stream, differentially-private federated aggregation, and a provable corpus-sampling bound. *Causal rigor* adds a stepped-wedge cluster trial, a synthetic-control study, a doubly-robust estimator, partial-identification and Manski attrition bounds, and a confirmatory pre-registration. *Autonomy* graduates from a single agent to a fleet: an orchestrator under a global safety budget, a runtime policy monitor proving no safety-automaton violation, online-learning guards, safe-interruptibility, and a liveness guarantee. *Adoption* targets standardization—an ISO-track certificate standard, multi-implementation conformance, IDE/CI/ORM/cloud integrations, and a neutral governance foundation—while the *research frontier* adds global-suite repair synthesis, verified learned heuristics, neuro-symbolic proof search whose every proof is machine-checked, and certified distributional robustness. The *artifact* layer is re-proven 2.0: one-command full reproduction, every-number provenance, a living related-work comparison, a negative-results registry, and a grand-unified evidence index 2.0 that cross-links all pillars into one falsifiable target.

Standard-scale certificate interchange. The newest interoperability slice makes the standardization claim executable rather than aspirational: Patchline includes PLCI/1, a line-oriented certificate interchange language with an ABNF grammar, canonical SHA-256 rule, real-code evidence digests, and independent Go, Rust, Python, and TypeScript checkers. Its gate accepts valid proof-carrying migration-safety certificates, rejects malformed verdicts, dangling evidence,

hash mismatches, and bad file digests, and fails if any implementation disagrees with the others. This is deliberately small enough to audit yet strong enough to serve as the seed of a multi-tool certificate ecosystem. The next gate turns parser interoperability into semantic exchange: Rails `strong_migrations`, Django migration checks, and Prisma Migrate diagnostics are projected into PLCI/1 certificates, parsed with real repository digest verification, and reconstructed back into analyzer-specific verdict projections. The proof is intentionally non-vacuous: only declared fields are preserved, the reconstruction uses the parsed certificate rather than the original fixture, and negative controls mutate risk, corrupt evidence, omit a preserved field, and submit an unsupported analyzer schema.

The standards-body conformance corpus raises that exchange layer from checker agreement to a citable reference suite. Four PLCI/1 examples cover `safe`, `guarded`, `blocked`, and `unsupported` verdict semantics over real Patchline source, proof, grammar, and documentation files. Each example pairs a positive certificate proof with a near-miss negative control whose syntax and file digests are valid but whose verdict/obligation semantics must be rejected; the expected checker output is Ed25519-signed and verified against a pinned public key. The gate also tampers with a signed reference output and proves the tamper is rejected, making the corpus suitable for external implementers rather than merely internal regression tests.

`make certificate-interop-dashboard-gate` turns that corpus into a public drift dashboard. The gate rebuilds a byte-preserving vector view of the frozen corpus, reruns the Go, Rust, Python, and TypeScript checkers with real repository evidence verification, and compares each checker's acceptance, rejection, verdict, risk score, certificate identifier, canonical hash, certificate bytes, and negative-control error text against the signed reference payload. It publishes both `dashboard.json` and `dashboard.md`; a tampered checker report must produce a `canonical_sha256` drift and a failing dashboard. This changes interoperability from "the implementations agreed once" to a release-time, reviewer-readable drift surface. `make conformance-failure-minimizer-gate` then turns the first disagreement back into a minimal debugging artifact: one checker, one corpus case, one vector kind, and one copied `witness.plci` certificate with reference-versus-observed deltas. The gate proves this on a tampered Python checker report and proves the clean-control path emits `no_failure` without a certificate witness.

The latest compatibility gate adds schema evolution to that story. Legacy PLCI/0 certificates use a coarser `risk-class` field; `make certificate-migration-gate` verifies their legacy canonical hashes, maps `low`, `medium`, `high`, and `unknown` into PLCI/1 basis-point risk scores, re-renders canonical PLCI/1 bytes, and then rechecks the migrated certificates with real file digest verification. The compatibility corpus migrates four legacy positives covering all verdict classes and rejects two legacy negatives (an unregistered risk class and a `safe` verdict with an assumed obligation), proving old verdicts remain checkable across schema revisions rather than being grandfathered by policy.

The new lifecycle gates make PLCI useful after issuance. `make certificate-normalizer-gate` canonicalizes order-insensitive witness sections before signing or comparison, re-parses the normalized bytes, and proves equivalent shuffled witnesses converge to the same identity. `make certificate-semantic-diff-gate` compares current or migrated certificates at the obligation level, reporting `strengthened`, `weakened`, `unchanged`, and conservative `changed` transitions while treating `refuted` as a counterexample state rather than a point on a confidence scale. `make certificate-revocation-gate` signs revocation and supersession records, binds each payload into the Merkle-style ledger, and rejects terminal-state rewrites. `make certificate-plugfest-gate` packages that lifecycle for external tools: submissions are validated offline by recomputing conformance, normalization, semantic diff, revocation replay, and reproducible log hashes in process, without executing the submitted checker.

14 Developer and Contributor Experience

A research artifact only earns stars if it is pleasant to extend. PATCHLINE ships a scaffolder (`scripts/new-ecosystem.sh`) that generates the example specification, worker script, gate script, and documentation stub for a new ecosystem following the established pattern, and an *onboarding quest*—a six-step path from scaffold to a passing real-repo gate that a contributor can complete in under one hour. A dedicated gate proves the scaffolder emits valid, runnable artifacts and that the quest narrative stays in sync with the required steps. PATCHLINE also exposes deterministic plugin interfaces (parser, fact extractor, linker, ranker, proposal generator, compare check, report renderer) and a `golden-fixture` command that turns a real-repo slice into a tiny deterministic Go test without vendoring the full repository.

15 Case Studies

`lobsters/lobsters` (Rails, `db/migrate`). On a pinned commit, PATCHLINE scans 158 files and surfaces 328 ranked data-change risks with 100 provenance slices, generating 8 review artifacts with zero deterministic checks failed. This is the headline landing demo and is itself gate-protected so the README's numbers cannot drift.

`apache/avro` (schema compatibility). PATCHLINE flags dozens of Avro record fields without defaults and multiple proto2 required fields as schema-evolution hazards, demonstrating that the same risk lens applies to serialization schemas, not just databases.

`helm/charts` (infra/data ordering). PATCHLINE classifies ten migration/database jobs as sequenced by explicit Helm hooks (including Kong and Anchore migrations) and three as unordered, showing that ordering risk is recoverable from real infrastructure manifests.

`harrydevforlife/building-lakehouse` (pipelines). A single repository exercises three pipeline frameworks at once—Airflow backfills, dbt table materializations, and Spark overwrite writes—and PATCHLINE records the destructive operations across all three as uniform pipeline facts.

`laaksomavrick/twitter-go` (NoSQL + minimization). PATCHLINE detects nine Cassandra changes including destructive `DROP KEYSPACE`/`DROP TABLE`, then delta-debugs a real migration to a 1-minimal reproduction.

16 Related Work

PATCHLINE sits at the intersection of several tool families but is distinguished from each. *Migration linters* (e.g. online-schema-change advisors, framework-specific guards) typically check a single statement or a single ecosystem; PATCHLINE unifies nine relational ecosystems with NoSQL, pipelines, schemas, and infrastructure under one fact model. *Code scanners* target injection, secrets, and CWE classes in application code; PATCHLINE targets the orthogonal axis of destructive data change and carries provenance for each finding. *Observability platforms* show what happened at runtime but do not statically rank data-change risk; PATCHLINE integrates their export formats to populate an independent runtime-evidence axis. *AI coding assistants* generate migrations and guards but cannot prove safety; PATCHLINE treats generation as bounded, quarantined, and subject to deterministic rejection. Finally, PATCHLINE's *gate methodology*—executable claims against

pinned public code with drift protection—is a reproducibility contribution in its own right. Table 12 summarizes the positioning.

Table 12: Positioning of PATCHLINE relative to neighboring tool families.

Property	**Migr. linter**	**Code scanner**	**Observability**	**Patchline**
Polyglot data surfaces	no	no	no	yes
Provenance per finding	partial	partial	yes	yes
Static risk ranking	partial	yes	no	yes
Runtime corroboration	no	no	yes	yes
Safe generation	no	no	no	yes
Reproducible proof gates	no	no	no	yes

17 Threats to Validity

Construct validity. PATCHLINE operationalizes "data-change risk" as a ranked set of facts with explicit destructiveness and blast-radius properties rather than as a single ground-truth label. To keep this construct honest, severity is calibrated against *independent* danger evidence (incident clusters, rollback/fix migrations, recurrences) rather than against PATCHLINE's own score, and the calibration lift is reported. The blinded three-rater false-positive adjudication, with Cohen's κ and three-way agreement, further guards against a definition of "risk" that only PATCHLINE agrees with.

Internal validity. Because every artifact is byte-stable for fixed inputs and every claim is regenerated by a gate against a pinned commit, the results reported here are not subject to run-to-run variance or silent environmental drift; a regression in code, documentation, or an upstream ref fails the corresponding gate loudly. The ablation study isolates each signal's contribution by removing provenance links, cross-file context, generated guards, runtime traces, and risk budgets one at a time, so the reported value of each component is measured rather than asserted.

External validity. The evaluation deliberately spans nine relational ecosystems plus NoSQL, pipeline, schema, and infrastructure surfaces, and stratifies by ecosystem, repository size, and migration age/change type with a real download proven from each stratum. This breadth mitigates, but does not eliminate, the risk that results are specific to the chosen public repositories; longitudinal reruns over multiple historical commits per repository test temporal stability.

Known blind spots. The seeded false-negative study explicitly measures what PATCHLINE misses by injecting known public-incident hazard analogues into real layouts and reporting recall, and the limitations ledger (Section 18) publishes the documented heuristic boundaries rather than hiding them.

18 Limitations

PATCHLINE is deliberately conservative and therefore has documented blind spots, which it publishes in a limitations ledger rather than hiding. Dynamic SQL assembled at runtime is only partially recovered; vendor-specific NoSQL aggregation pipelines are matched heuristically rather than fully parsed; dynamically generated DAGs and templated SQL are detected only when literals are present; cross-file Helm/Argo dependencies are not yet linked into a single ordering graph; and per-message protobuf field-number uniqueness is not yet diffed across versions. Source provenance for very large monorepos fetched over Mercurial/Fossil without a native binary is approximated by content hashing. PATCHLINE ranks and explains risk; it does not certify a migration as safe, and it intentionally avoids full-repair claims.

19 Discussion

Why a fact layer instead of a model. A recurring question is why PATCHLINE does not simply fine-tune a model to classify risky diffs. The answer is auditability. A reviewer approving a destructive migration must be able to see *why* a change was flagged, trace it to a specific line, and reproduce the judgment tomorrow. A fact—kind, path, rationale, typed identifiers, destructiveness flag—is a unit of explanation, not just a unit of classification. The uniform envelope also means that adding a new surface (say, a sixth NoSQL engine) extends coverage without changing how ranking, diffing, or reporting work, which is what keeps the system tractable as breadth grows.

Why gates instead of unit tests. Unit tests pin behavior against fixtures the author chose; gates pin behavior against *real, pinned public repositories* the author did not control, and additionally assert that the documentation and README still describe the behavior. This catches a class of failures unit tests cannot: silent capability drift, documentation that overstates the tool, and upstream changes that invalidate a claim. The cost is that gates are heavier and require network access to a pinned slice; PATCHLINE mitigates this with content-addressed caching and an offline mode so that gate sweeps are fast and repeatable.

Generalization beyond data changes. While PATCHLINE targets data-change risk, the architecture—deterministic stages, a provenance-carrying fact layer, bounded quarantined generation, and gate-backed claims—is domain-agnostic. The plugin interfaces (parser, fact extractor, linker, ranker, proposal generator, compare check, report renderer) make it plausible to retarget the same machinery at other high-stakes, weak-oracle domains such as access-control changes or feature-flag rollouts. We leave such retargeting to future work but note that nothing in the core couples it to databases specifically.

On the cost of conservatism. Principle P3 (conservative by construction) means PATCHLINE accepts false negatives to avoid confident false positives. This is the right trade for a tool whose output gates a human reviewer's attention: a noisy analyzer is quickly ignored, whereas a quiet, high-precision analyzer that occasionally misses a hazard still raises the floor on review quality. The seeded false-negative study exists precisely to keep this trade visible and measurable rather than implicit.

20 Future Work

The completed roadmap now extends beyond the original 700-step artifact into standard-scale certificate interoperability, source-free live feedback ingestion, gate-bound drift-aware threshold updates, reviewer-outcome counterfactual logging for previous releases, and operational learning that remains shadowed, local, frozen, calibrated, retained, and trust-regressed before it influences policy. PATCHLINE therefore already combines deterministic analysis, bounded generation, real-repo gates, formal and causal scaffolding, autonomy safety cases, standards hooks, privacy-preserving reviewer-outcome aggregation, and long-horizon impact evidence. The remaining frontier is not another isolated detector; it is the transition from a generational artifact to shared infrastructure. The open roadmap items around this transition include: production-grade remediation loops for rollback, repair escrow, and patch-series safety; a public evidence marketplace with licenses, bounties, minimization, and governance; engine-version semantics for production databases, locks, replication, partitioning, and online schema change; socio-technical accountability for appeals, fairness, ethics, and detector deprecation; education and certification materials that make migration safety teachable; resilient offline, edge, confidential, and disaster-recovery deployments; and an award-to-standard transition that externalizes benchmarks, errata, conformance labels, and long-term stewardship. These items are intentionally recorded as future work, not as claims in the deployed gate catalog: the standard for adding them remains the same as the current system's standard—a positive proof on real code, a negative control, stable artifacts, and documentation that fails if it overstates the evidence.

21 Conclusion

PATCHLINE demonstrates that the highest-risk changes teams ship—destructive, polyglot, cross-cutting data changes—can be analyzed *deterministically*, with full provenance, across relational, NoSQL, pipeline, schema, and infrastructure surfaces, and that generation can be made safe by bounding it, quarantining it, and deterministically rejecting it. Just as important, PATCHLINE shows that *every* such claim can be made reproducible: over 600 gates prove the system against real, pinned public code and fail loudly on drift, and a research-grade evaluation methodology—blinded adjudication, seeded recall, ablations, effect sizes, and severity calibration—turns the artifact into evidence. We believe this combination of breadth, safety, and gate-backed reproducibility is a template for trustworthy program-analysis artifacts.

Availability. PATCHLINE is a single Go binary with no required external services. Every result in this paper is regenerated by a `make` target against pinned public repositories.

A A Worked End-to-End Example

To make the pipeline concrete, we walk through a single invocation against a pinned slice of the `lobsters/lobsters` Rails application. The command requests the full pipeline with a bounded budget and template-only generation:

```
go run ./cmd/patchline repo analyze \
  -github lobsters/lobsters \
  -ref 3b80b47aa5aaba37ec44413e7d1dc96fcf1585b6 \
  -subpath db/migrate \
  -stages inventory,baseline,propose,compare \
```

```
-proposal-kind all \
-budget files=8,lines=100,tokens=12000,changes=2 \
-no-llm -out results/generated/landing-demo -json
```

Fetch. PATCHLINE resolves the pinned commit, downloads the `db/migrate` slice, and writes `source.json` recording the GitHub host, the resolved 40-character commit, and the archive SHA-256. Because the commit is pinned, every subsequent run is reproducible.
Inventory. PATCHLINE scans 158 files, identifies the Rails migration system and its native command, and emits the fact stream. Schema-evolution facts are derived directly from the migration bodies—no pre-authored schema is required—so an `ALTER TABLE ... DROP COLUMN` becomes a `schema_evolution` fact carrying the table and column as typed identifiers.
Baseline. PATCHLINE ranks 328 data-change risks, attaches 100 provenance slices, estimates blast radius, and minimizes proof holes. The output is written both as `baseline.json` and as `baseline.sarif` for IDE and CI ingestion.
Propose and compare. Within the budget, PATCHLINE generates 8 review artifacts (guards and tests) under `patchline-proposals/`, all non-executable. The `compare` stage re-analyzes them; in this run zero deterministic checks fail, and the accept/reject decision for each artifact is recorded with its evidence delta. The headline numbers—158 files, 328 risks, 100 provenance slices, 8 artifacts, 0 failed checks—are themselves asserted by the `landing-readme-gate` so the README cannot drift from reality.

B Fact Schema Reference

Every fact shares the schema in Table 13. Detectors differ only in the `kind` and the `properties` they populate; the uniform envelope is what allows ranking, diffing, and tabulation to be detector-agnostic.

Field	Meaning
`version`	Fact schema version, enabling forward-compatible evolution.
`id`	Stable content-derived identifier for cross-referencing (e.g. from a trace span).
`kind`	The detector category, e.g. `schema_evolution`, `nosql_change`, `data_pipeline_change`, `schema_compatibility`, `infra_data_ordering`.
`path`	Repository-relative path of the evidence.
`confidence`	`observed` for directly matched signals, `derived` for cross-fact inferences.
`rationale`	Human-readable justification; the basis of PATCHLINE's explainability.
`identifiers`	Typed identifiers (table, model, engine, operation, job, ...) for linking.
`properties`	Detector-specific key/values, including the `destructive` or `breaking` flag.

Table 13: The uniform fact envelope shared by all detectors.

C Detector Rule Catalog

Tables 14–15 list representative rules for the pipeline and schema-compatibility detectors. Each rule is signal-gated (P3): the framework or format signal must match before any rule is evaluated, which is what keeps prose and unrelated code from being misclassified.

Table 14: Data-pipeline detector: signals and representative operations.

Framework	Signal	Destructive operations
Airflow	`from airflow`, `DAG(`	embedded DROP/TRUNCATE, backfill, `clear_task_instances`, dagrun reset
dbt	`{{ config(`, `{{ ref(`	`-full-refresh`, `materialized='table'`
Spark	`pyspark`, `SparkSession`	`.mode("overwrite")`, `saveAsTable`, `insertInto`
Kafka	`KafkaConsumer`, `@KafkaListener`	`auto.offset.reset`, `seekToBeginning`, `-reset-offsets`

Table 15: Schema-compatibility detector: formats and breaking risks.

Format	Inspected	Breaking risk
protobuf	message fields, syntax level	proto2 `required` field; message with no `reserved`
Avro	record fields	field without a `default`
registry	`buf.yaml`, `buf.gen.yaml`	compatibility-policy presence

D Infrastructure/Data Ordering Markers

The ordering analysis (Section 5) treats the markers in Table 16 as evidence that a data-change job is sequenced relative to its rollout. A migration or database job co-located with any such marker is classified *sequenced*; one with none is *unordered*.

Table 16: Deploy-ordering markers correlated with migration/database jobs.

Marker	Source
`helm.sh/hook` (pre/post-install, pre/post-upgrade)	Helm
`argocd.argoproj.io/sync-wave`	Argo CD
`initContainers`, `depends-on`	Kubernetes
`depends_on`, `wait = true`, `atomic = true`	Terraform / Helm provider

E Selected Gate Catalog

PATCHLINE ships more than 600 gates; Table 17 lists a representative selection grouped by theme to convey the breadth of what is proven against real, pinned public code. Each gate is a single `make` target.

Theme	**Representative gates**
Source breadth	`mercurial-fossil-source-gate`, `monorepo-boundary-gate`
Migration breadth	`multi-ecosystem-migration-gate`, `nosql-change-gate`, `data-pipeline-gate`, `schema-compat-gate`, `infra-ordering-gate`
Intervention safety	`generated-code-quarantine-gate`, `rejected-generated-gate`, `intervention-regression-archive-gate`, `expand-contract-templates-gate`, `staged-backfill-planner-gate`, `canary-validation-gate`, `repair-risk-escrow-gate`, `incident-postmortem-importer-gate`, `verified-rollback-planner-gate`, `remediation-cost-optimizer-gate`, `reviewability-examples-gate`
Runtime evidence	`otel-trace-gen-gate`, `datadog-timeline-gate`, `prom-grafana-gate`, `runtime-confidence-gate`
Evaluation rigor	`fp-adjudication-gate`, `fn-discovery-gate`, `ablation-study-gate`, `effect-size-strata-gate`, `severity-calibration-gate`
Stratification	`ecosystem-balanced-benchmark-gate`, `repository-size-stratification-gate`, `migration-age-stratification-gate`, `longitudinal-public-reruns-gate`
Security & privacy	`secret-scan-gate`, `archive-security-gate`, `threat-model-gate`, `redaction-stability-gate`, `privacy-metrics-gate`, `live-feedback-ingestion-gate`, `feedback-retention-lifecycle-gate`, `supply-chain-provenance-gate`
Robustness & DX	`fuzz-coverage-gate`, `parser-dashboard-gate`, `fixture-minimizer-gate`, `onboarding-quest-gate`, `performance-budget-gate`
Artifact & paper	`reviewer-walkthrough-gate`, `artifact-evaluation-kit-gate`, `paper-figures-gate`, `paper-appendix-gate`, `camera-ready-checklist-gate`
Formal methods	`invariant-extract-gate`, `symexec-gate`, `modelcheck-gate`, `causal-graph-gate`, `counterfactual-gate`

Theme	Representative gates
Decision science	risk-economics-gate, uncertainty-calibration-gate, active-learning-gate, reviewer-sim-gate, live-feedback-ingestion-gate, drift-threshold-update-gate, reviewer-counterfactual-log-gate, safe-online-evaluation-gate, adopter-active-learning-gate, policy-freeze-gate, live-calibration-monitor-gate, human-trust-regression-gate, live-learning-methodology-gate
Release & interop	rc-rehearsal-gate, leaderboard-gate, dossier-gate, interop-gate, standards-body-conformance-corpus-gate, certificate-interop-dashboard-gate, conformance-failure-minimizer-gate, certificate-migration-gate, certificate-normalizer-gate, certificate-semantic-diff-gate, certificate-revocation-gate, certificate-plugfest-gate
Generalization	framework-holdout-gate, perturbation-robustness-gate, historical-replay-study-gate, external-replication-kit-gate
Trust stack	signed-provenance-chain-gate, reproducible-build-attestation-gate, merkle-audit-log-gate, sbom-pinned-deps-gate, red-team-adversarial-gate
Reviewer & community	quickstart-sixty-seconds-gate, inline-review-surface-gate, ci-pr-bot-gate, governance-policy-gate, citation-doi-gate
Research frontier	learned-risk-model-gate, neuro-symbolic-verdict-gate, theorem-prover-backend-gate, abstention-policy-gate, causal-effect-estimate-gate
Camera-ready	repro-appendix-gate, dataset-datasheet-gate, model-card-gate, evaluation-preregistration-gate, camera-ready-build-gate
Mechanized foundations	mechanized-operational-semantics-gate, soundness-theorem-gate, completeness-characterization-gate, refinement-types-columns-gate
Ecosystem scale	prevalence-estimator-gate, corpus-harness-gate, longitudinal-public-reruns-gate, drift-monitor-gate
Adoption & impact	self-serve-onboarding-gate, policy-as-code-layer-gate, causal-effect-estimate-gate, grand-unified-evidence-index-gate

Theme	Representative gates

Table 17: A representative selection from the 600+-gate catalog, grouped by theme.

F Security and Privacy Model

Because PATCHLINE ingests *untrusted* repositories and archives, it carries an explicit threat model that is itself gate-verified. Archive extraction defends against path traversal, symlink escape, malformed archives, and archive bombs, while still extracting pinned public GitHub archives through a content-addressed cache. Generated proposal files are non-executable, unapplied, and excluded from native execution unless the user explicitly enables an allowlist of safe checks. Redaction is byte-stable across repeated and resumed runs, secret-scanning proves that redacted reports, prompts, bundles, and CI artifacts do not leak deterministic canary values, and an aggregate privacy-metrics mode emits source-free risk trends with salted cohort identifiers so results can be shared without raw evidence. The canary-validation protocol applies that discipline to local production-like samples: generated reports contain only snapshot hashes, row hashes, and cell hashes, not sampled values or redaction salts. The source-free live-feedback ingester applies the same discipline to adopter reviewer outcomes: it stores no source, diffs, paths, raw evidence, finding IDs, evidence hashes, adopter IDs, or salts, and low-count detector groups are folded into a dimension-free residual bucket. Counterfactual replay over previous releases consumes only those published groups, excludes raw confidence values, and reports decile-boundary ambiguity rather than reconstructing private individual outcomes. The live-learning gates extend this privacy model: local active-learning queues are explicitly non-shareable, shareable aggregates exclude local case identifiers, incident freezes name only source-free organization cohorts and audited releases, calibration alerts operate over published deciles, and retention reports use a fixed as-of date rather than a wall clock. Binaries, release archives, and public-corpus downloads carry deterministic provenance and sorted checksums.

G Reproduction Guide

Every claim in this paper is regenerated by a `make` target. To reproduce the breadth results of Section 5 against pinned public repositories:

```
make multi-ecosystem-migration-gate   # Laravel + 7-framework matrix
make nosql-change-gate       # Cassandra DROP on twitter-go + 5-engine matrix
make data-pipeline-gate      # Airflow+dbt+Spark on building-lakehouse
make schema-compat-gate      # Avro + protobuf on apache/avro
make infra-ordering-gate     # Kong/Anchore sequencing on helm/charts
make fixture-minimizer-gate  # 1-minimal reproductions per ecosystem
make parser-dashboard-gate   # coverage + fuzz robustness dashboard
make certificate-interop-dashboard-gate   # PLCI checker drift dashboard
make certificate-migration-gate # PLCI/0 verdicts recheck as PLCI/1
make certificate-plugfest-gate  # offline PLCI lifecycle submission replay
make live-feedback-ingestion-gate # source-free reviewer outcome aggregates
make drift-threshold-update-gate # gate-bound advisory calibration updates
make reviewer-counterfactual-log-gate   # previous-release replay without raw outcomes
make safe-online-evaluation-gate   # shadow detectors until gates pass
make adopter-active-learning-gate   # local examples, aggregate uncertainty
```

```
make policy-freeze-gate      # high-stakes incident pinning
make live-calibration-monitor-gate   # decile drift alerts
make feedback-retention-lifecycle-gate   # expiry/anonymization proof
make human-trust-regression-gate   # explanation faithfulness checks
make live-learning-methodology-gate   # recall gain without over-reliance
```

Each target downloads only the pinned slice it needs, runs the real analyzer with `-no-llm`, asserts its invariants, and writes a `gate-summary.json`. A failing assertion—whether from a code regression, a documentation drift, or a changed upstream ref—fails the target loudly, which is precisely the property that makes the artifact trustworthy.